

1、程序代码

```
#include <bits/stdc++.h>
using namespace std;

class Solution
{
public:
    int search(vector<int>& nums, int target)
    {
        int len = nums.size();
        int l = 0, h = len - 1, mid = (l + h) >> 1;

        while (l < h)
        {
            mid = (l + h) >> 1;
            if (nums[mid] == target)
                return mid;
            else if (nums[mid] < target)
            {
                l = mid + 1;
            }
            else
                h = mid - 1;
        }
        return nums[l] == target ? l : -1;
    }
};

int main()
{
    int t;
    cin >> t;
    while(t-->
    {
        int n, target;
        cin >> n;
        vector<int> nums(n);
        for (int i = 0; i < n; i++)
        {
            cin >> nums[i];
        }
        cin >> target;

        Solution s;
        int res = s.search(nums, target);

        if (res != -1)
        {
            cout << "目标值 " << target << " 在数组中的索引为: " << res << endl;
        }
        else
        {
            cout << "目标值 " << target << " 不在数组中" << endl;
        }
    }
    return 0;
}
```

2、运行结果

```
2
6
-1 0 3 5 9 12
9
目标值 9 在数组中的索引为：4
6
-1 0 3 5 9 12
2
目标值 2 不在数组中

Process returned 0 (0x0)    execution time : 18.220 s
Press any key to continue.
```

3、实习总结与体会

二分查找算法，用于在一个整数数组 `nums` 中查找目标值 `target`。可以有效降低时间复杂度

(1) 二分查找需要序列有序，将要查找的元素 `target` 与当前查找范围最中间的元素 `mid` 比较。如果 `target` 小于 `mid`，查找范围变为当前范围的前半段；如果 `target` 大于 `mid`，查找范围变为当前范围的后半段。不断重复上述操作，直到剩余查找范围区间长度为1。

(2) 确定查找的初始边界，即数组的起始索引 `l` 和结束索引 `h` 很重要

(3) 通过 `(l + h) >> 1` 的方式计算数组的中间索引 `mid`。`>>` 是右移运算符，相当于除以2。

(4) 在循环结束后，检查 `nums[l]` 是否等于 `target`，如果是则返回 `l`，否则返回 `-1`，表示目标值不在数组中。